



01 · APERTURA

Ingeniería de Software I

INGENIERÍA DE SOFTWARE I

UNIDAD II – MODELOS DE DESARROLLO DE SOFTWARE Y DESARROLLO RÁPIDO DEL SOFTWARE

Prof. Lic. Guillermo Jacobo González Rodas Mst. PMP



02 · OBJETIVOS

Objetivos

01

Entender diferencias entre métodos tradicionales y ágiles.

02

Conocer principios, prácticas y limitaciones de los MAs.

03

Entender cómo un enfoque impacta en la calidad.

04

Analizar condiciones bajo las cuales los MAs son efectivos.



03 · RECORRIDO

Contenido

Modelos Tradicionales

- Cascada
- Prototipación
- Fases
- Espiral
- RUP

Métodos Ágiles

- Manifiesto Ágil
- ASD
- Scrum
- XP



04 · CONTEXTO

Ingeniería de Software

La calidad, metodologías y madurez de entornos siguen siendo temas de discusión.

Q

Calidad

C

Costos

T

Plazos



05 · ESENCIA

Dificultades en Producción de Software

Esencia: complejidad, conformidad, invisibilidad y cambios.

01

Complejidad

Estructuras lógicas inherentemente complejas.

02

Conformidad

Adaptarse a reglas y estándares.

03

Invisibilidad

Estructuras no visibles.

04

Cambios

Requisitos cambian constantemente.



06 · DEFINICIÓN

Proceso de Software

«Conjunto relacionado de actividades y tareas implicadas en el desarrollo y evolución de un sistema software»

— Sommerville

Sommerville, I. Ingeniería de Software, 7ª ed. Addison Wesley, 2005.

¿**Por qué?** Organizar trabajo, reducir riesgos.

¿**Qué define?** Actividades, métodos, herramientas.



07 · CLASIFICACIÓN

Modelos de Procesos de Software

Tradicionales

- Cascada
- Prototipación
- Fases
- Espiral
- RUP

Ágiles

- ASD
- Scrum
- XP



08 · TRADICIONAL

Modelo en Cascada

El más antiguo: debe completarse un estado antes de comenzar el siguiente.

Características:

- Secuencial y documentado.
- Útil para visualizar el proceso.

Problema:

- No refleja la realidad.
- Errores se detectan tarde.



09 · EN LA PRÁCTICA

Modelo en Cascada

01 Análisis

02 Diseño Sistema

03 Diseño Programas

04 Codificación

05 Testing

06 Aceptación

07 Mantención



10 · PROTOTIPO

Modelo de Prototipación

Ventajas:

- Construcción rápida.
- Visión común usuario-desarrollador.
- Reduce riesgo.

Ideal cuando:

- Requisitos no claros.
- Retroalimentación temprana.



11 · FLUJO

Modelo de Prototipación

01 Req. Prototipo

02 Diseño

03 Implementación

04 Testing

Ciclo: Se repite hasta alcanzar los requisitos del sistema.



12 · INCREMENTAL

Modelo de Desarrollo en Fases

El mercado no acepta grandes retardos; se desarrolla en fases.

Estrategia: Entregar por partes, funcionalidad desde el inicio.

Ejemplo: Fase 1: 20%, Fase 2: 60%, Fase 3: 100%.



13 · EJEMPLO

Modelo de Desarrollo en Fases

Fase	Funcionalidad	Avance*
Fase 1	Básica	20%
Fase 2	Adicional	60%
Fase 3	Completa	100%

* Valor aproximado



14 · ITERATIVO

Modelo en Espiral

Combina desarrollo con **Análisis de Riesgo**.

01 Planificación

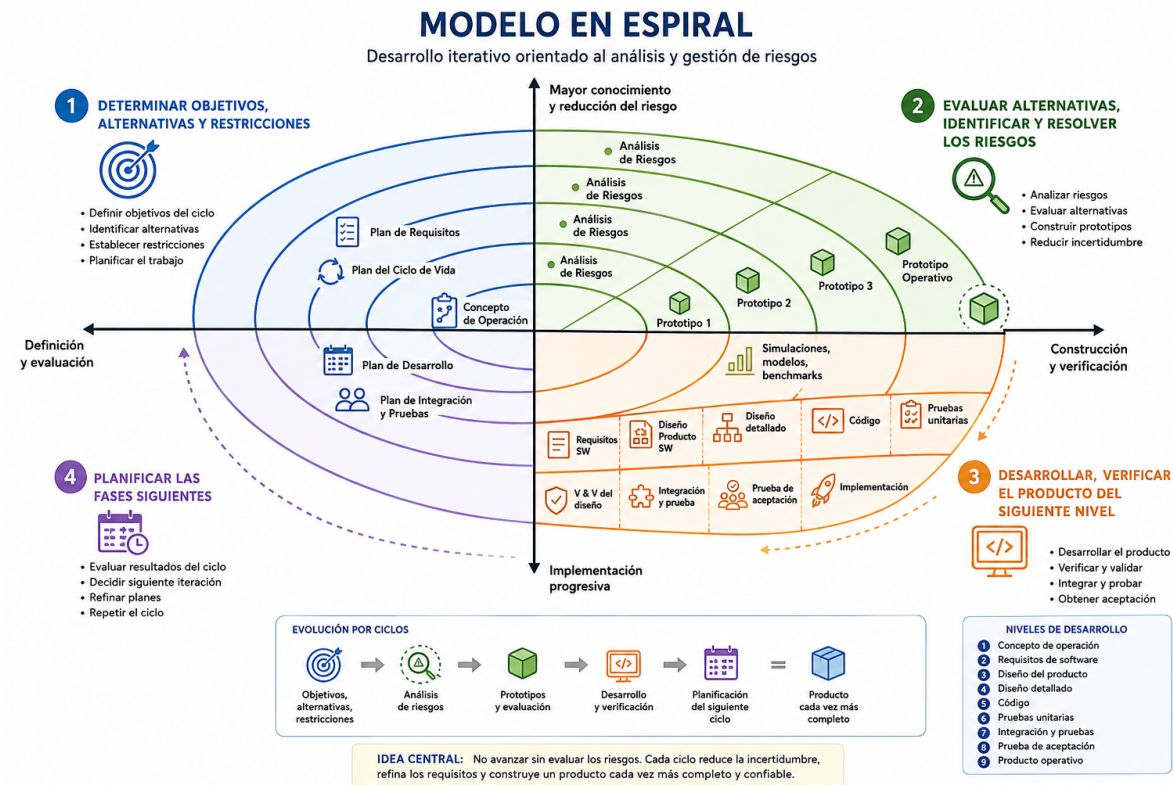
02 Análisis Riesgo

03 Ingeniería

04 Evaluación

15 · VISUALIZACIÓN

Modelo en Espiral





16 · FRAMEWORK

RUP – Rational Unified Process

Framework para describir procesos de desarrollo específicos.

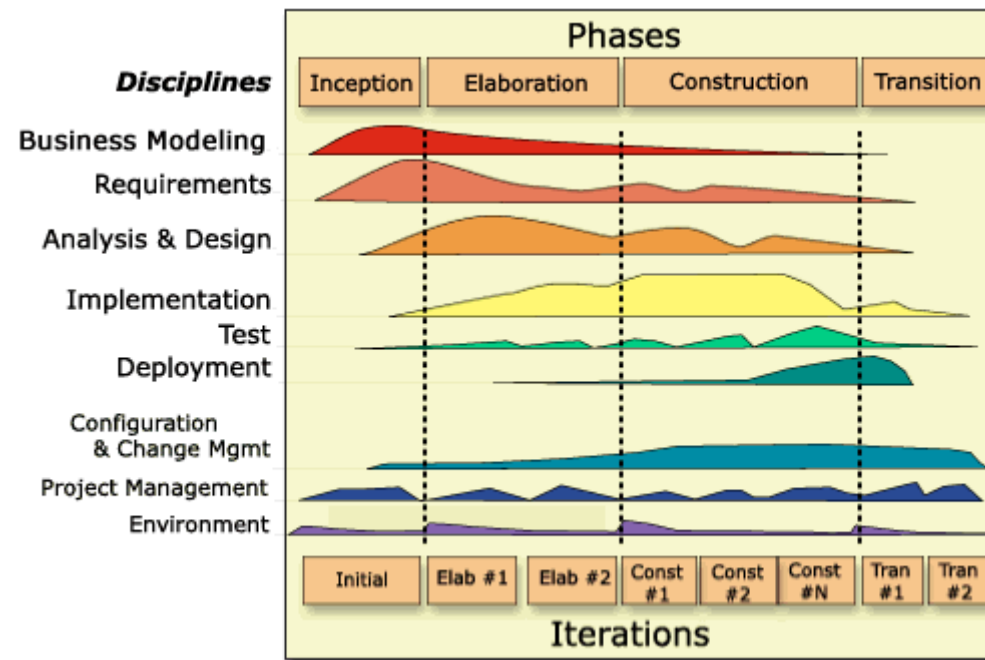
Jacobson, I., Booch, G., Rumbaugh, J. The Unified Software Development Process. Addison-Wesley, 1999.

4 fases: Concepción, Elaboración, Construcción, Transición.

Características: Iterativo, incremental, orientado a arquitectura.

17 · DIAGRAMA

RUP – Rational Unified Process





18 · LIMITACIONES

Problemas de los Modelos Tradicionales

01

Incumplimiento de plazos.

02

Reducción de funcionalidades.

03

Difícil adaptación a cambios.

04

Excesiva documentación.



19 · ORIGEN

Métodos Ágiles (MAs)

En febrero de 2001, en Utah-EEUU, nace el término "Ágil" aplicado al desarrollo de software.

Beck, K. et al. Manifiesto for Agile Software Development. agilemanifesto.org, 2001.

17 expertos de la industria buscaron alternativas a los problemas de los modelos tradicionales.



20 · DEFINICIÓN

Métodos Ágiles

Estrategias que promueven prácticas adaptativas, en lugar de predictivas.

Pressman, R. Ingeniería del Software: Un enfoque práctico, 7ª ed. McGraw-Hill, 2010. Cap. 2.

Principio clave: Responder al cambio en vez de seguir un plan.



21 · MANIFIESTO

Métodos Ágiles

Individuos e interacciones sobre procesos y herramientas

Software funcionando sobre documentación extensiva

Colaboración con el cliente sobre negociación contractual

Respuesta al cambio sobre seguir un plan



22 · INDIVIDUOS

Métodos Ágiles

Al individuo y las interacciones sobre el proceso y las herramientas.

La gente es el principal factor de éxito. Es más importante un buen equipo que las mejores herramientas.



23 · SOFTWARE

Métodos Ágiles

Desarrollar software que funciona más que conseguir documentación.

No producir documentos a menos que sean necesarios para una decisión importante.



24 · CLIENTE

Métodos Ágiles

La colaboración con el cliente más que la negociación de un contrato.

Interacción constante entre cliente y equipo; la colaboración guía el desarrollo.



25 · CAMBIO

Métodos Ágiles

Responder a los cambios más que seguir estrictamente un plan.

La habilidad de responder a cambios es más importante que seguir un plan prefijado.



26 · EQUILIBRIO

Agilidad vs. Proceso

CMM/CMMI y XP pueden complementarse.

Hallazgos: La agilidad no elimina la necesidad de proceso.

Desafío: Encontrar equilibrio entre agilidad y proceso.



27 · REFLEXIÓN

Interrogantes

¿Resuelven los MAs todos los problemas?

¿Son aplicables a todo tipo de desarrollo?

¿Cuánta agilidad? ¿Cuánto proceso?

Desafío: encontrar equilibrio.



28 · PRINCIPALES

Metodologías Ágiles

ASD

Adaptive Software Development

XP

Programación Extrema

Scrum

Marco de trabajo ágil



29 · ASD

Adaptive Software Development (ASD)

Desarrollado por J. Highsmith en 2000.

Highsmith, J. Adaptive Software Development: A Collaborative Approach to Managing Complex Systems. Addison-Wesley, 2000.

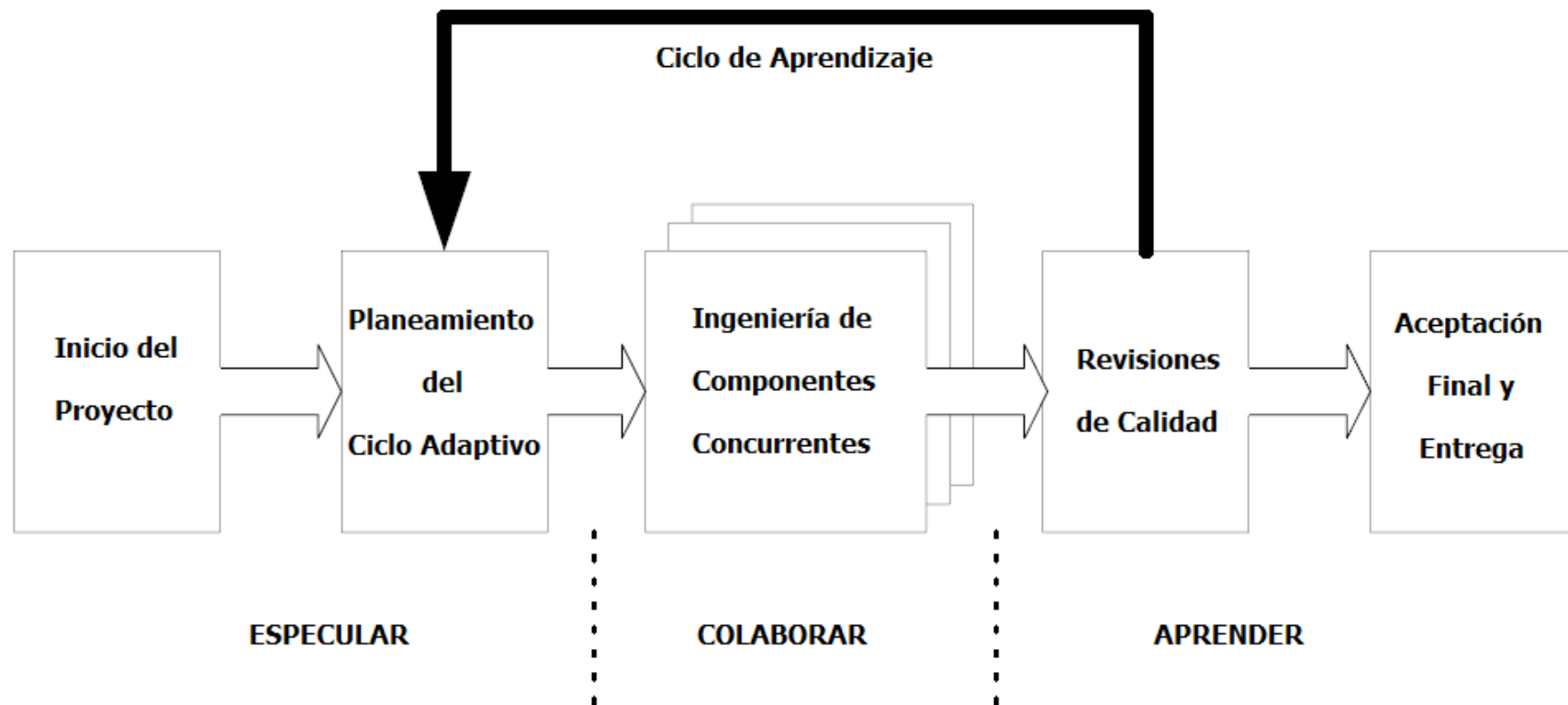
«Balancearse en el borde del Caos»

— Idea fundamental

Buscar equilibrio entre creatividad y estructura.

30 · DIAGRAMA

Proceso del ASD





31 · APLICACIÓN

Uso actual del ASD

Casi no se utiliza porque no especifica prácticas detalladas.

Debe complementarse con otra metodología que defina prácticas concretas.



32 · INTRODUCCIÓN

Programación Extrema (XP)

Desarrollado por K. Beck en 1999.

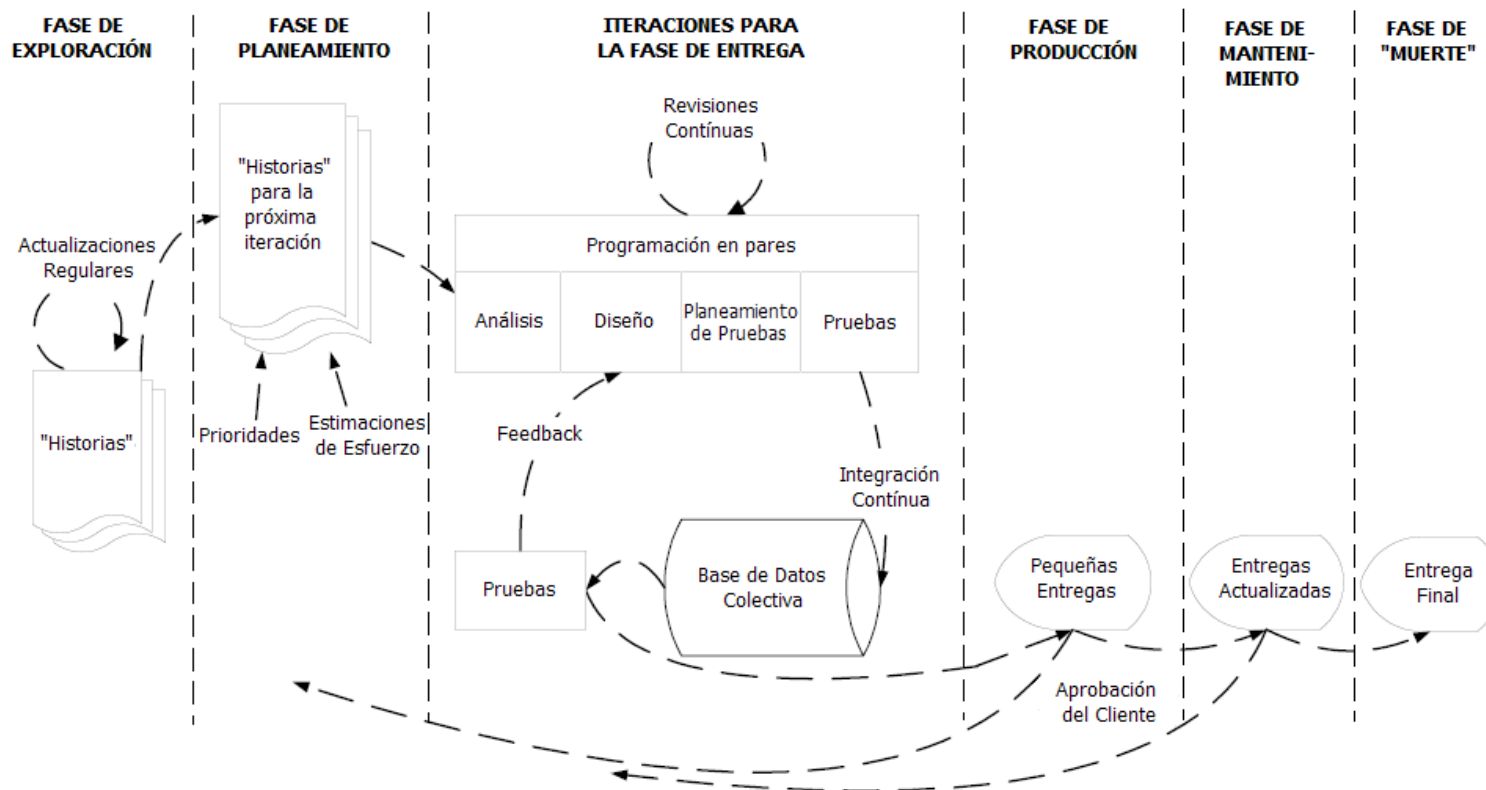
Beck, K. Extreme Programming Explained: Embrace Change. Addison-Wesley, 1999.

Probablemente la metodología ágil más conocida y utilizada.

XP combina prácticas conocidas de manera innovadora.

33 · DIAGRAMA

Programación Extrema (XP)





34 · PRÁCTICAS

Prácticas de Programación Extrema

01

Parejas

Dos programadores, una estación.

02

Diseño Simple

Lo más simple que funcione.

03

Refactorización

Mejorar sin cambiar comportamiento.

04

Testing Continuo

Pruebas antes del código.



35 · PRÁCTICAS

Prácticas de Programación Extrema

05

Integración Continua

Integrar varias veces al día.

06

Propiedad Colectiva

Cualquiera cambia cualquier código.

07

Código Estándar

Todos programan igual.

08

Sistema Metafórico

Historia común del sistema.



36 · MERCADO

Uso actual de XP

El 38% del mercado ágil utiliza XP por sus prácticas bien definidas.



37 · INTRODUCCIÓN

SCRUM

La metodología más utilizada por equipos ágiles.

Schwaber, K., Sutherland, J. The Scrum Guide. Scrum.org, 2020.

Trabajo en equipo con resultado incremental.



38 · ACTIVIDADES

SCRUM - ACTIVIDADES

01

Sprint

2-8 semanas.

02

Planning

Planificar sprint.

03

Daily

Reunión diaria.

04

Review

Revisar resultado.

05

Retrospective

Mejorar proceso.



39 · HERRAMIENTAS

SCRUM - HERRAMIENTAS

Product Backlog

Lista priorizada.

Sprint Backlog

Trabajo del sprint.

Burn Down

Gráfico progreso.



40 • ROLES

SCRUM - ROLES

Product Owner

Define y prioriza.

Scrum Master

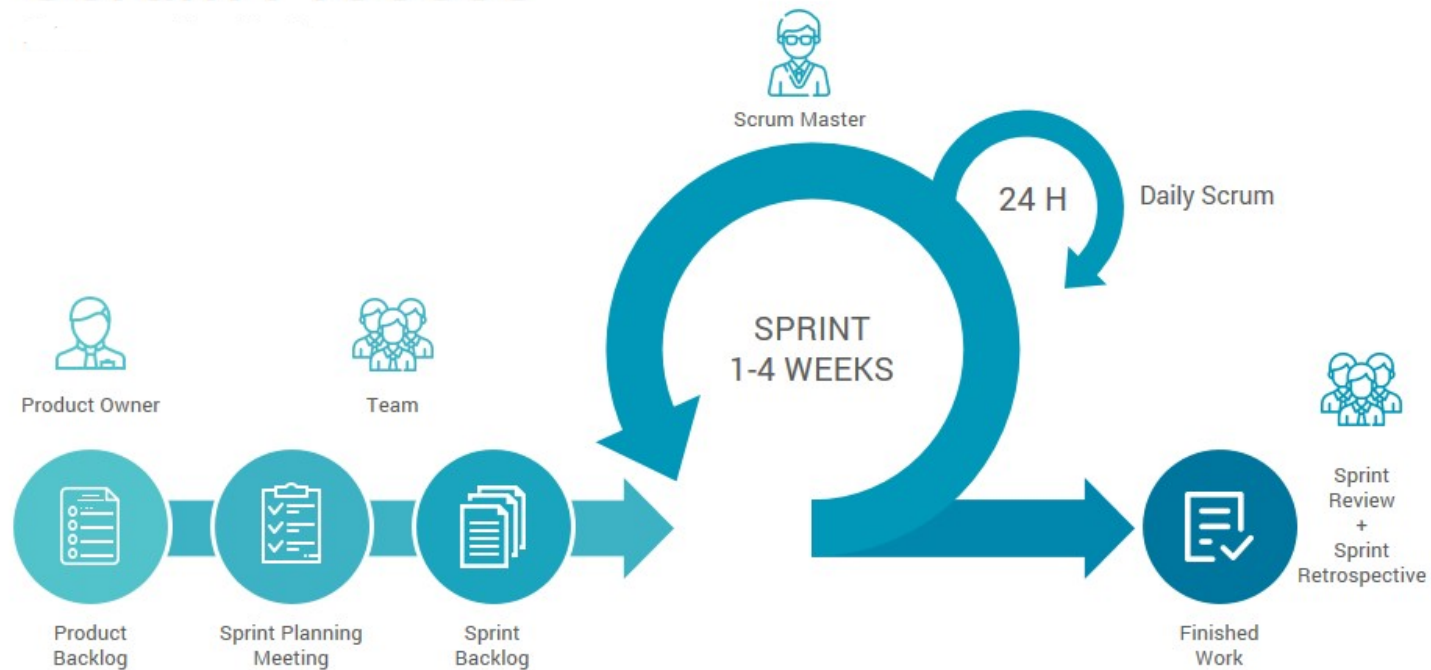
Facilita y remueve obstáculos.

Team

Equipo de desarrollo.

41 · DIAGRAMA
SCRUM

Scrum Process





42 · MERCADO

Uso actual de SCRUM

Scrum es muy utilizado; se integra con XP para gestión + desarrollo.



43 · FORTALEZAS

Ventajas de las MAs

01

Iteraciones cortas para correcciones rápidas.

02

Ciclos de 2 a 8 semanas.

03

Adaptable a nuevos riesgos.

04

Entrega continua de valor.



44 • LIMITACIONES

Desventajas de las MAs

01

Puede llevar al caos.

02

Difícil mantener interés del cliente.

03

Requiere equipo comprometido.

04

No adecuado para requisitos estables.



45 · SÍNTESIS

Métodos Ágiles

Los MAs son una respuesta a las limitaciones de los modelos tradicionales.

No son la solución universal, pero aportan herramientas valiosas.



46 · CIERRE

Conclusiones

Las MAs proveen un marco aplicable a entornos sin procesos definidos.

Aportan flexibilidad y adaptabilidad.

El desafío es encontrar el equilibrio entre agilidad y proceso.



47 • CIERRE

Muchas gracias

MUCHAS GRACIAS

PREGUNTAS